

Figure 7.8 Anatomy of the `whitespace` analyzer

Listing 7.13 The `whitespace` tokenizer in action

```
POST _analyze
{
  "text":"Peter_Piper picked a peck of PICKLED-peppers!!",
  "analyzer": "whitespace"
}
```

If we test this script, we get this set of tokens:

```
["Peter_Piper", "picked", "a", "peck", "of", "PICKLED-peppers!!"]
```

Note these two points from the result: first, the text was tokenized only on whitespaces, not dashes, underscores, or punctuation. Second, case is preserved—the capitalization of the characters and words was kept intact.

As mentioned earlier, similar to the `simple` analyzer, the `whitespace` analyzer is not exposed to configurable parameters. If we need to modify the analyzer's behavior, we may need to create a modified custom version. We examine custom analyzers shortly.

7.3.4 The keyword analyzer

As the name suggests, the `keyword` analyzer stores text without any modifications or tokenization. That is, the analyzer does not tokenize the text, nor does the text undergo any further analysis via filters or tokenizers. Instead, it is stored as a string representing a `keyword` type. As figure 7.9 shows, the `keyword` analyzer is composed of a `noop` (no-operation) tokenizer and no character or token filters.

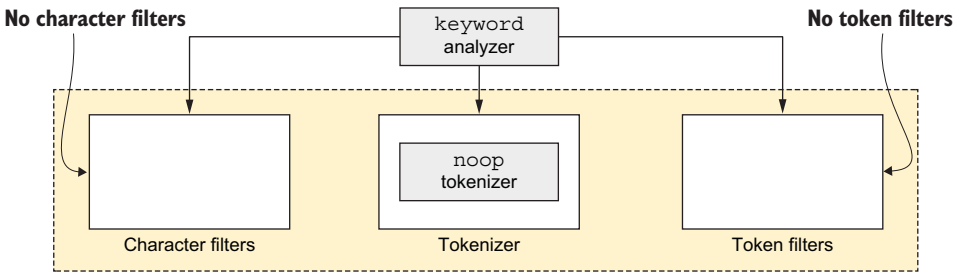


Figure 7.9 Anatomy of the `keyword` analyzer